



Projecto de Programação com Objectos 18 de Setembro de 2026

Esclarecimento de dúvidas:

- Consultar sempre o corpo docente atempadamente: presencialmente ou através do endereço **po-tagus@disciplinas.tecnico.ulisboa.pt**.
- Não utilizar fontes de informação não oficialmente associadas ao corpo docente (podem colocar em causa a aprovação à disciplina).
- Não são aceites justificações para violações destes conselhos: quaisquer consequências nefastas são da responsabilidade do aluno.

Requisitos para desenvolvimento, material de apoio e actualizações do enunciado, Ver informação completa na secção **[Projecto]** no Fénix:

- O material de apoio é de uso obrigatório e não pode ser alterado.
- Verificar atempadamente (mínimo de 48 horas antes do final de cada prazo) os requisitos exigidos pelo processo de avaliação.

Processo de avaliação:

- O projecto é constituído por três entregas: (i) A primeira corresponde à modelação do domínio da solução; (ii) a segunda corresponde à concretização de uma parte do domínio da solução e de algumas funcionalidades da aplicação; e (iii) a terceira parte corresponde à concretização do domínio total da solução e das restantes funcionalidades da aplicação.
- Após a entrega 3ª parte do projecto, haverá a realização de um teste prático **individual**. O teste prático é realizado **em papel e sem acesso a um computador**.
- Ver informação completa nas secções **[Projecto]** e **[Método de Avaliação]** da página Web da disciplina no Fénix (incluindo as datas para cada uma das entregas do projecto e do teste prático).
- O material de apoio é de uso obrigatório e não pode ser alterado. Não serão consideradas quaisquer alterações aos ficheiros de apoio disponibilizados: eventuais entregas dessas alterações serão automaticamente substituídas durante a avaliação da funcionalidade do código entregue.
- Trabalhos não presentes no Fénix no final do prazo têm classificação 0 (zero) (não são aceites outras formas de entrega).
- A avaliação do projecto pressupõe o compromisso de honra de que o trabalho foi realizado pelos alunos correspondentes ao grupo de avaliação.
- **Fraudes na realização do projecto terão como resultado a exclusão dos alunos implicados no processo de avaliação.**

O objectivo do projecto é desenvolver um sistema de gestão de entregas rápidas de produtos (ERP — Express Retail Platform). Este documento está organizado da seguinte forma. A secção 1 apresenta as entidades do domínio da aplicação a desenvolver. A secção 2 descreve os requisitos de desenho que a aplicação desenvolvida deve oferecer. A secção 3 descreve, forma resumida, a arquitectura genérica que a solução deve seguir. A interação com o utilizador da aplicação a desenvolver está descrita nas secções 4 e 5.

Neste texto, o tipo **negrito** indica um literal (i.e., é exactamente como apresentado); o símbolo $_$ indica um espaço; e o tipo *italico* indica uma parte variável (i.e., uma descrição).

1 Entidades do Domínio

Esta secção descreve as várias entidades que vão ser manipuladas no contexto da aplicação a desenvolver. Existem vários conceitos importantes neste contexto: empresas, produtos, clientes, cabazes, compras, encomendas e entregas. Os conceitos listados não são os únicos possíveis no modelo e as suas relações (assim como relações com outros conceitos não mencionados) podem depender das escolhas do projecto. Os clientes e os produtos são independentes das empresas: existem por si e podem relacionar-se com várias empresas. Cada empresa, no entanto, mantém registo dos produtos que disponibiliza e das compras feitas. Uma compra representa a aquisição de produtos por um cliente a uma empresa e resulta de finalizar um cabaz. Uma compra pode dar origem a uma ou mais encomendas, de acordo com a estratégia de divisão de encomendas utilizada. Cada encomenda pode ser enviada de acordo com uma determinada modalidade e pode incluir serviços adicionais que alteram o seu custo e/ou tempo de entrega.

1.1 Produto

O sistema mantém um registo de todos os produtos. Cada produto é identificado por um número de produto. O identificador é atribuído automaticamente e incrementalmente (a partir de 1 ou do último valor atribuído, caso o estado do sistema tenha sido recuperado). Todos os produtos têm: um nome/descrição (cadeia de caracteres não vazia); um preço base (número inteiro positivo); uma categoria; um prazo de entrega base (número inteiro de dias); e a proveniência.

O preço base é uma propriedade intrínseca do produto e é definido ao nível do sistema. O preço base é independente das empresas que disponibilizam o produto. O sistema utiliza o preço base para calcular o preço recomendado do produto. O preço

recomendado representa o valor de referência do produto no sistema. O seu cálculo depende do tipo de produto e de outras informações disponíveis no sistema. O produto existe independentemente das empresas que o disponibilizam. Assim, o mesmo produto pode ser disponibilizado por várias empresas, sendo possível que cada uma pratique um preço de venda diferente. Cada produto tem uma categoria, de acordo com a sua natureza. Inicialmente, consideram-se as seguintes categorias: (i) alimentar; (ii) electrónica; e (iii) vestuário. Deve ser possível adicionar novas categorias com impacto mínimo na aplicação já desenvolvida.

1.1.1 Proveniência de Produtos

Os produtos podem ter diferentes proveniências. Inicialmente, consideram-se as seguintes: (i) produto nacional; e (ii) produto importado. Um produto nacional possui apenas as propriedades gerais dos produtos. Por seu lado, um produto importado possui, para além das propriedades gerais dos produtos, o país de origem (cadeia de caracteres não vazia) e a taxa alfandegária (número real expressando uma percentagem).

O preço recomendado é calculado de forma diferente consoante o tipo de produto. O cálculo do preço recomendado deve ser realizado de forma polimórfica, de modo a que o código que utiliza produtos não tenha de conhecer o tipo concreto de cada produto. Deve ser possível introduzir ou especializar proveniências com impacto mínimo na concretização desenvolvida.

1.1.2 Produto Nacional

Para um produto nacional, o preço recomendado depende do preço base e do número total de unidades actualmente disponíveis desse produto nas empresas que o disponibilizam. Se o número total de unidades disponíveis for:

- superior a 10000, o preço recomendado corresponde a 90
- inferior a 100, o preço recomendado corresponde a 130
- nos restantes casos, o preço recomendado corresponde ao preço base.

O cálculo do preço recomendado não é arredondado, nem precisa de o ser, pois o resultado tem, no máximo, uma casa decimal: 130% de um preço base de 45 dá 58.5 (apresentado como 58.50). Por exemplo, considerando um produto nacional com preço base de 100 euros, se o número total de unidades disponíveis for:

- igual a 50, o preço recomendado é 130 euros;
- igual a 5000, o preço recomendado é 100 euros;
- igual a 12000, o preço recomendado é 90 euros;

1.1.3 Produto Importado

Para um produto importado, o preço recomendado corresponde ao preço base corrigido com taxa alfandegária correspondente. Se a taxa alfandegária for $t\%$, o preço recomendado é calculado de acordo com $\text{preço base} \times (100 + t)/100$. Por exemplo, considerando um produto importado com preço base de 100 euros e uma taxa alfandegária de 20.0%, o preço recomendado será: $100 \times (100 + 20)/100 = 120.0 \text{ euros}$. Como a taxa é um número real, o preço recomendado pode ter mais de duas casas decimais: com um preço base 7 e uma taxa 12.5%, é $7 \times 112.5/100 = 7.875$. O valor não é arredondado — só o preço de venda o é; é apenas apresentado com duas casas decimais (7.88).

1.2 Empresa

O sistema mantém um registo de todas as empresas. Cada empresa é identificada por um número de empresa. O identificador é atribuído automaticamente e incrementalmente (a partir de 1 ou do último valor atribuído, caso o estado do sistema tenha sido recuperado). Cada empresa tem ainda um nome (cadeia de caracteres não vazia).

Uma empresa pode disponibilizar vários produtos e cada produto pode ser disponibilizado por várias empresas. Para cada produto que disponibiliza, a empresa deve manter informação sobre o número de unidades disponíveis (valor inteiro) e o factor multiplicativo do preço recomendado (número positivo). O número de unidades disponíveis e o factor multiplicativo podem ser diferentes para cada empresa que disponibiliza o mesmo produto.

O factor multiplicativo permite que uma empresa pratique um preço de venda diferente do preço recomendado pelo sistema. O preço de venda de um produto pela empresa é calculado multiplicando o preço recomendado do produto pelo factor multiplicativo definido pela empresa. Assim, duas empresas podem disponibilizar o mesmo produto a preços diferentes e com quantidades diferentes em stock. Por exemplo, considere o produto P_1 , com um preço recomendado de 100 euros, disponibilizado pela empresa A com um factor multiplicativo de 0,9, e pela empresa B com um factor multiplicativo de 1,2. A empresa A disponibiliza P_1 com o preço de 90 euros enquanto que a empresa B disponibiliza o mesmo produto com um preço de 120 euros.

1.2.1 Alterações de Inventário

O inventário de um produto numa empresa pode ser alterado através da especificação de dois valores: um número inteiro e um número real. O número inteiro é somado ao número de exemplares disponíveis do produto nessa empresa: se for positivo, aumenta-o; se for negativo, diminui-o, desde que o número de exemplares não fique negativo. O número real é o novo factor multiplicativo do produto nessa empresa, que tem de ser positivo. O inventário só é actualizado se a quantidade e o factor multiplicativo forem ambos válidos; caso contrário, a operação não tem qualquer efeito. É também por esta via que uma empresa passa a disponibilizar um produto já registado no sistema: nesse caso, a quantidade e o factor têm de ser ambos positivos, porque uma empresa não passa a disponibilizar um produto com zero exemplares.

Um produto não é removido do sistema quando o seu inventário numa empresa chega a zero. Quando o inventário de um produto numa empresa chega a zero, o produto deixa de estar disponível nessa empresa, mas continua a existir no sistema e pode continuar a ser disponibilizado por outras empresas. Se o inventário de um produto numa empresa passar de zero para um valor positivo, o produto volta a estar disponível nessa empresa.

O número total de unidades disponíveis de um produto corresponde à soma dos inventários desse produto em todas as empresas que o disponibilizam. Esta informação é utilizada, nomeadamente, no cálculo do preço recomendado dos produtos nacionais.

1.3 Clientes

O sistema mantém um registo de todos os clientes. Cada cliente é identificado pelo seu endereço de correio electrónico, que deve ser único. Cada cliente tem ainda informação sobre o seu nome. Estes campos não podem ser vazios.

1.4 Cabaz de Compras

Os produtos que um cliente pretende adquirir a uma empresa são acumulados num cabaz. Cada cliente tem um cabaz por cada empresa em que compra. Os cabazes fazem parte do estado persistente da aplicação. O cabaz é constituído por um conjunto de itens, onde cada item corresponde a um produto disponibilizado por essa empresa e a uma quantidade. Acrescentar ao cabaz um produto que já lá esteja acumula as quantidades. Quantidades negativas diminuem o total, que nunca é negativo: se o total ficar nulo — ou se a quantidade a subtrair ultrapassar a que lá está — o produto é removido do cabaz. É esta a forma de retirar um produto do cabaz. Os cabazes fazem parte do estado persistente da aplicação.

Acrescentar exemplares — seja incluindo um produto novo, seja aumentando a quantidade de uma linha que já existe — só é aceite se a quantidade resultante dessa linha não for superior ao número de exemplares que a empresa associada ao cabaz tem do produto. Caso contrário, o cabaz não é alterado. Retirar exemplares é sempre aceite, desde que o produto esteja no cabaz.

O cabaz pode dar origem a uma compra caso o cliente decida comprar os produtos presentes no cabaz. O cabaz regista ainda as opções que serão aplicadas à compra:

- a estratégia de divisão em encomendas;
- a modalidade de transporte;
- o conjunto de serviços adicionais.

A modalidade e os serviços adicionais escolhidos aplicam-se a todas as encomendas resultantes da compra. Enquanto não for finalizado, o cabaz não tem qualquer efeito no resto do sistema: em particular, não reserva nem consome inventário, e os preços apresentados são sempre os preços de venda praticados pela empresa nesse momento. Ter sido possível colocar no cabaz uma quantidade de um produto não garante, portanto, que ela ainda esteja disponível quando a compra for finalizada: a empresa pode, entretanto, ter vendido exemplares desse produto a outros clientes. Ao finalizar o cabaz e apenas nesse momento:

- os preços de venda são fixados nas linhas da compra;
- a compra é dividida em encomendas de acordo com a estratégia configurada no cabaz;
- todas as encomendas recebem a modalidade de transporte e os serviços adicionais registados no cabaz;
- o inventário dos produtos na empresa é decrementado e o cliente é debitado;
- o cabaz é esvaziado: fica sem linhas, sem modalidade de transporte e sem serviços adicionais, e a estratégia de divisão volta à de omissão. Nada da compra acabada de fazer transita para a seguinte.

Um cabaz vazio não pode ser finalizado, nem o pode ser um cabaz em que ainda não tenha sido escolhida a modalidade de transporte. Se, no momento de finalizar, a empresa já não tiver exemplares suficientes de algum dos produtos do cabaz (por exemplo, porque o inventário foi entretanto alterado), a operação falha sem qualquer efeito e o cabaz mantém-se inalterado.

1.5 Compras

O sistema mantém o registo de todas as compras. Cada compra é identificada por um número de compra, atribuído automaticamente e incrementalmente (a partir de 1 ou do último valor atribuído, caso o estado do sistema tenha sido recuperado). Uma compra é sempre feita por um único cliente a uma única empresa numa determinada data.

Uma compra é constituída por um conjunto de linhas, cada uma descrevendo um produto e informação correspondente, e resulta do fecho de um cabaz. Cada linha contém o seguinte:

- um produto;
- uma quantidade (número de exemplares adquiridos desse produto);
- um preço unitário (o preço de venda do produto pela empresa no momento em que o cabaz é fechado/finalizado).

O preço unitário utilizado numa compra deve ser guardado na própria linha da compra. Alterações posteriores ao preço base, ao preço recomendado, ao factor multiplicativo da empresa, ou a qualquer outra propriedade utilizada no cálculo do preço, não devem alterar o valor de compras já efectuadas.

O preço de venda praticado por uma empresa é o preço recomendado do produto multiplicado pelo factor dessa empresa, arredondado a duas casas decimais:

```
//_variable_names_are_merely_descriptive  
unitPrice=_Math.round(recommendedPrice*_multiplier*_100)/_100.0;
```

Este é o **único** arredondamento do sistema: nenhum outro valor é arredondado — nem o preço recomendado, nem os totais. O seu propósito é a previsibilidade: o produto do preço recomendado pelo factor pode ter uma expansão decimal longa (por exemplo, 58.5×1.1), e arredondá-lo dá um valor com apenas duas casas decimais, a partir do qual tudo o resto se calcula. Todos os valores monetários são apresentados com duas casas decimais, sem excepção — os que são inteiros, como o preço base, as tarifas e os custos dos serviços e das encomendas (15.00, 312.00), e os que têm mais casas, como um preço recomendado de 7.875 (7.88). Trata-se apenas de apresentação: o valor guardado, e usado nos cálculos, não é alterado. O separador decimal é o ponto.

O custo de uma linha é obtido multiplicando o preço unitário do produto pela quantidade. O custo total dos produtos da compra corresponde à soma dos custos de todas as suas linhas. Uma compra pode dar origem a uma ou mais encomendas, de acordo com uma estratégia de divisão de encomendas. A divisão de uma compra em várias encomendas pode ter impacto no preço final a pagar.

1.5.1 Divisão das Compras em Encomendas

Cada encomenda tem um custo de processamento de 5 euros. Assim, uma compra dividida em várias encomendas pode ter um custo superior ao de uma compra enviada numa única encomenda. Se uma encomenda tiver um valor superior a 50 euros (somatório dos vários itens presentes na encomenda), o custo de processamento é zero. O prazo de entrega base de uma encomenda é igual ao maior prazo de entrega base dos vários produtos presentes na encomenda.

O custo total de processamento da compra corresponde à soma dos custos de processamento das suas encomendas. A este custo podem acrescer ainda os custos associados ao método de transporte seleccionado e aos serviços adicionais. O custo de uma encomenda inclui: (i) o seu custo de processamento; (ii) o custo adicional da modalidade de transporte aplicada à encomenda; e (iii) o custo dos serviços adicionais aplicados à encomenda. O custo total de uma compra corresponde, assim, ao custo dos produtos acrescido do custo das suas encomendas.

A forma como uma compra é dividida em encomendas é determinada por uma estratégia de divisão de encomendas. Inicialmente, devem ser suportadas, pelo menos, as seguintes estratégias: encomenda única (UNICA) e agrupamento por prazo de entrega base dos produtos (PRAZO). Deve ser possível introduzir novas estratégias de divisão de compras em encomendas com impacto mínimo na concretização desenvolvida.

Encomenda única Todos os produtos da compra são colocados numa única encomenda. Neste caso, a compra origina uma única encomenda, independentemente dos prazos de entrega base dos produtos.

Agrupamento por prazo de entrega As linhas de uma compra são agrupadas de acordo com os seus prazos de entrega base dos produtos correspondentes, de acordo com o procedimento a seguir descrito:

1. calcula-se a média dos prazos de entrega base dos produtos das linhas da compra;
2. todas as linhas cujo produto tem prazo de entrega base menor ou igual à média são agrupadas numa única encomenda;

3. cada linha cujo produto tem prazo estritamente maior do que a média é enviada separadamente, numa encomenda separada.

Por exemplo, considere-se uma compra com quatro itens cujos produtos têm os prazos de entrega base 3 (P1), 8 (P2), 2 (P3) e 8 (P4). A média dos prazos é $(3 + 8 + 2 + 8)/4 = 5,25$. Logo, P1 e P3 (com prazos de entrega base menores ou iguais a 5,25) seguem juntos na encomenda A com um prazo de entrega base igual a 3 e P2 e P4 (prazo $> 5,25$) seguem em encomendas separadas B e C (ambas com um prazo de entrega base igual a 8), de acordo com o método acima.

Embora a estratégia de divisão que origina várias encomendas possa permitir que alguns produtos sejam entregues mais cedo, também pode resultar num custo superior devido ao custo adicional associado a cada encomenda.

1.5.2 Modalidades de Transporte

Cada encomenda é enviada utilizando uma modalidade de transporte. Inicialmente, devem ser suportadas, pelo menos, as seguintes modalidades: normal (NORMAL) e expresso (EXPRESSO). Uma modalidade de transporte é a designação de um nível de serviço. O custo e o prazo não lhe estão fixados no código: para cada modalidade são definidos: (i) uma tarifa de transporte (número inteiro); (ii) e um tempo de transporte (número inteiro de dias). A tarifa e o tempo podem ter valores diferentes consoante a modalidade de transporte. Por exemplo, a modalidade de transporte NORMAL pode ter uma tarifa de 3 euros e um tempo de transporte igual a 3 dias enquanto que a modalidade de transporte EXPRESSO tem uma tarifa de 8 euros e um tempo de transporte igual a 1 dia.

Por razões de simplificação, a modalidade de transporte é definida no cabaz e aplica-se a todas as encomendas da compra. No entanto, o modelo deve permitir que cada encomenda seja enviada utilizando uma modalidade diferente. A escolha da modalidade afecta tanto o custo como a data prevista de entrega da encomenda. Uma compra só pode ser finalizada depois de escolhida a modalidade de transporte. Deve ser possível introduzir novas modalidades de transporte com impacto mínimo na concretização desenvolvida.

1.5.3 Serviços Adicionais

Quando está a finalizar o processo de compra de um cabaz, o cliente pode seleccionar serviços adicionais, podendo aumentar o custo de cada encomenda, assim como o tempo necessário para a sua preparação e entrega. Os serviços são escolhidos no cabaz e aplicam-se a todas as encomendas da compra. O modelo deve, no entanto, permitir que cada encomenda tenha os seus próprios serviços. Cada serviço tem um identificador único, e implica um custo e tempo adicional a cada encomenda. Os serviços iniciais a considerar são os seguintes:

- **Embrulho:** Tem o identificador `EMBRULHO`, e um custo e tempo adicional de 2 euros e 1 dia, respetivamente.
- **Cartão de oferta:** Tem o identificador `CARTAO_OFERTA`, e um custo e tempo adicional de 1 euro e 0 dias, respetivamente.

Os serviços adicionais podem ser combinados: uma encomenda pode incluir simultaneamente embrulho, cartão de oferta ou outro serviço adicional. Inclusive, deve ser possível adicionar o mesmo serviço várias vezes. Depois de adicionar um serviço, não é necessário removê-lo. Deve ser possível introduzir novos serviços adicionais sem alterar o código das encomendas ou dos serviços já existentes.

1.6 Data de Entrega das Encomendas

Cada encomenda tem uma data prevista de entrega. A data prevista de entrega é determinada considerando a data da compra, o prazo de entrega base da encomenda, o tempo de transporte da modalidade seleccionada e o tempo adicional introduzido pelos serviços seleccionados. Uma encomenda é considerada entregue quando a data actual atinge ou ultrapassa a sua data prevista de entrega.

1.7 Gestão de Tempo

A unidade de tempo do sistema é o dia. A data do sistema começa no dia 1 (um) e faz parte do estado persistente do sistema. A data do sistema pode ser avançada. Sempre que a data é alterada, devem ser verificadas as encomendas existentes para determinar: (i) quais as encomendas que serão entregues no dia seguinte; (ii) quais as encomendas que ainda não tinham sido entregues e cuja data prevista de entrega foi atingida ou ultrapassada.

Quando a data actual atinge ou ultrapassa a data prevista de entrega de uma encomenda, esta considera-se entregue. A gestão do tempo deve ainda desencadear notificações (ver secção 1.8) referentes a encomendas a entregar no dia seguinte e encomendas entregues.

Por razões de simplificação, para garantir que as notificações são enviadas no dia correcto e pela ordem correcta, avançar N dias deve ser executado como avançar a data 1 dia N vezes. Caso N não seja um número positivo, então a operação de avançar data não tem qualquer efeito.

1.8 Notificações

O sistema deve notificar os clientes em duas situações: (i) um dia antes da data prevista de entrega, informando o cliente de que a encomenda será entregue no dia seguinte; (ii) no dia da entrega, informando o cliente de que a encomenda foi entregue.

Cada cliente pode escolher o mecanismo através do qual pretende receber notificações do sistema. O mecanismo de notificação pode variar de cliente para cliente. Por exemplo, um cliente pode receber notificações por correio electrónico, enquanto outro pode recebê-las através de WhatsApp. Por razões de simplificação, inicialmente, o sistema deve suportar um mecanismo de notificações (o mecanismo por omissão) em que cada cliente guarda directamente as suas notificações e estas depois podem ser escritas na saída do programa quando o cliente quiser. No entanto, deve ser possível introduzir novos mecanismos de notificação sem necessidade de alterar o código dos clientes ou do sistema responsável por gerar as notificações. As notificações devem ser enviadas pela ordem em que os acontecimentos ocorrem no sistema.

1.9 Serialização

É possível guardar e recuperar o estado actual da aplicação utilizando o mecanismo de serialização do Java, preservando toda a informação relevante do domínio da aplicação. Após a recuperação do estado, os identificadores atribuídos a novas entidades devem continuar a ser atribuídos incrementalmente a partir do último valor utilizado em cada caso. A aplicação não permite manter várias versões do estado da aplicação em simultâneo.

A serialização Java usa as classes da package `java.io`, em particular, a interface `java.io.Serializable` e as classes de leitura `java.io.ObjectInputStream` e escrita `java.io.ObjectOutputStream` (entre outras).

2 Requisitos de Desenho

O sistema a desenvolver deve seguir o princípio de desenho *aberto-fechado* por forma a aumentar a extensibilidade do seu código. Por exemplo, deve ser possível adicionar novos tipos de produto com um impacto mínimo no código já existente da aplicação. Devem ser possíveis extensões ou alterações de funcionalidade com impacto mínimo no código já produzido para a aplicação. O objectivo é aumentar a flexibilidade da aplicação relativamente ao suporte de novas funções. Assim, deve ser possível:

- Adicionar novos tipos de produto (e.g., produtos digitais ou perecíveis), incluindo a respectiva regra de cálculo do preço recomendado;
- Adicionar novas categorias de produto;
- Introduzir novas estratégias de divisão de compras em encomendas;
- Introduzir novas modalidades de transporte;
- Introduzir novos serviços adicionais, sem alterar o código das encomendas nem o dos serviços já existentes;
- Introduzir novos mecanismos de notificação, sem alterar o código dos clientes nem o do sistema que gera as notificações;
- Nenhuma entidade é removida pelas operações descritas neste enunciado (em particular, um produto cujo inventário numa empresa chegue a zero continua a existir no sistema). Ainda assim, a aplicação deve estar preparada para que a remoção de qualquer entidade possa vir a ser suportada.

3 Arquitectura da Aplicação

A aplicação a desenvolver deve seguir uma arquitectura em camadas: as entidades relacionadas com interacção com o utilizador estão concretizadas na camada de apresentação (package `erp.app`) e as entidades relacionadas com as entidades do domínio estão concretizadas na camada de domínio (package `erp.core`). Por forma a garantir uma independência entre as várias camadas da aplicação não deve haver código de interacção com o utilizador no código respeitante ao domínio da aplicação (concretizada na camada de domínio). Desta forma, será possível reutilizar o código do domínio da aplicação com outras concretizações da interface com o utilizador sem que isso implique alterar o código da camada de domínio. Para garantir um melhor desenho da aplicação toda a lógica de negócio deve estar concretizada no código respeitante ao domínio da aplicação (camada *core*) e não na camada de apresentação. Assim potencia-se a reutilização de código e, além disso, o código fica concretizado nas entidades a que diz respeito, o que torna a aplicação mais legível e mais fácil de manter.

A interacção com o utilizador deve ser realizada utilizando a *framework* de interacção com o utilizador **po-uilib**, disponível na secção **Projecto** da página da disciplina. A interacção com o utilizador suportada por esta framework é realizada através de menus, em que cada menu tem um determinado conjunto de opções. Cada menu é concretizado por uma subclasse da classe `pt.tecnico.po.ui.Menu` (fornecida pela framework), onde se deve indicar qual o conjunto de opções do menu. Cada opção

de um menu (designado como *comando*) é concretizada por uma subclasse de `pt.tecnico.uilib.menus.Command` (também fornecida pela framework) que tem a responsabilidade de suportar a funcionalidade correspondente à opção em causa. Todos os menus suportam automaticamente a opção **Sair** (fecha o menu).

As operações de pedido e apresentação de informação ao utilizador **devem** realizar-se através do próprio comando e do objecto *display* presente em cada comando, respectivamente. No caso de um comando utilizar mais do que um formulário para pedir informações ao utilizador então será necessário criar explicitamente pelo menos uma instância de `Form` durante a execução do comando em causa. As mensagens a apresentar ao utilizador normalmente são produzidas pelos métodos de interfaces já definidas na camada de apresentação do esqueleto da aplicação fornecido (ver secção 3.1).

Podem acontecer situações de erro durante a execução de um comando. Quando isto ocorre, o comando não deve ter qualquer efeito no estado da aplicação (excepto se indicado em contrário na operação em causa) e o comando deve instanciar uma excepção (pertencente a uma subclasse de `pt.tecnico.uilib.menus.CommandException` correspondente ao erro que aconteceu) e lançá-la de seguida. Estas excepções serão depois tratadas automaticamente pelo object `Menu` no contexto do qual o comando que lançou a excepção está a ser executado.

Antes de começara a desenvolver a aplicação, deve ler com atenção a documentação sobre a arquitetura da aplicação a aplicar no projeto e sobre a framework de interacção com o utilizador. A documentação está presente na secção *Material de Apoio -> Desenvolvimento de aplicações em Programação com Objectos* da página da disciplina.

3.1 Esqueleto da Aplicação

Já é fornecido um esqueleto da aplicação a desenvolver, não sendo por isso necessário concretizar a aplicação de raiz. Este esqueleto está disponível no arquivo `erp-skeleton.jar` presente na secção *Projecto* da página da cadeira. O esqueleto inclui a classe `erp.app.App`, que representa o ponto de entrada da aplicação a desenvolver e os vários comandos e menus da aplicação a desenvolver (descritos na secção 4). Alguns dos comandos já estão completamente finalizados, outros (a grande maioria) estão parcialmente concretizados e têm de ser alterados por forma a suportarem a funcionalidade descrita neste enunciado. Cada menu e respectivos comandos estão definidos num subpackage de `erp.app`. Por exemplo, o menu principal e respectivos comandos estão definidos em `erp.app.main`. Os menus já estão concretizados.

Cada subpackage relacionado com um menu contém ainda as interfaces `Message`, `Prompt` (definem as mensagens a utilizar na interacção com o utilizador) e `Label` (define as etiquetas do menu e dos vários comandos). Estes ficheiros **não** podem ser alterados. Potenciais omissões devem ser esclarecidas com o corpo docente antes de qualquer concretização.

O esqueleto inclui ainda algumas classes do domínio da aplicação (presentes no package `erp.core`) parcialmente concretizadas: a classe `Erp` (que representa o sistema ERP), a classe `ErpManager` (e que representa o ponto de entrada utilizado pelo código de interacção com o utilizador para realizar a funcionalidade pretendida) e a classe `Basket` (que representa um cesto de compras). Será ainda necessário concretizar novas entidades do domínio da aplicação e finalizar as que já estão parcialmente fornecidas por forma a ter um conjunto de entidades que suportem a totalidade da funcionalidade do domínio da aplicação a desenvolver.

De um modo geral, sempre que no contexto de uma operação com o utilizador, ou seja no contexto de um comando, aconteça algum *erro* (por exemplo, indicação de um identificador inválido de uma entidade do domínio), então a execução do comando em causa não deve ter qualquer efeito no estado da aplicação, excepto em caso de indicação contrária na operação em causa. Normalmente, nesta situação, o comando deve terminar com o lançamento de uma excepção associada ao tipo de erro que aconteceu. As excepções a lançar no contexto de um comando já estão definidas no package `erp.app.exception` do esqueleto da aplicação. Cada uma destas excepções representa uma situação de erro distinta e deve ser lançada por um comando sempre que essa situação acontece. Estas excepções serão depois tratadas de forma automática pela biblioteca de interacção *po-uilib* no contexto da classe `Menu`. Estas excepções **não** podem ser alteradas e apenas devem ser utilizadas no código de interacção com o utilizador pela razão indicada anteriormente. O package `erp.core.exception` contém algumas excepções a utilizar na camada do domínio, mas podem (e **devem**) ser acrescentadas novas excepções para representar outras situações anómalas que possam acontecer no contexto do domínio.

4 Interação com o Utilizador

Esta secção descreve a **funcionalidade máxima** da interface com o utilizador. Em geral, os comandos pedem toda a informação antes de proceder à sua validação (excepto onde indicado).

A aplicação permite gerir a informação sobre as várias entidades do modelo. Possui ainda a capacidade de preservar o seu estado (não é possível manter várias versões do estado da aplicação em simultâneo). No início, a aplicação está vazia, mas pode ser carregada uma base de dados textual com conceitos pré-definidos. Neste caso, a aplicação começará com um estado referente às entidades que foram carregadas no arranque da aplicação. Deve ser possível registar, visualizar, e operar sobre as várias entidades do domínio. Deve ser possível efectuar pesquisas sujeitas a vários critérios e sobre as diferentes entidades geridas pela aplicação.

A descrição da funcionalidade de cada comando indica quais as excepções que podem ser lançadas pelo comando. Para cada interacção com o utilizador que exista no contexto de um comando em que é necessário utilizar uma mensagem específica é

indicado qual é o método da interface `Prompt` (caso a mensagem seja para pedir dados) ou `Message` (caso a mensagem seja para apresentar dados) que é responsável por devolver a mensagem em questão.

No contexto de um comando pode ser pedido ao utilizador um ou mais identificadores de objectos do domínio necessários para a realização da funcionalidade do comando em causa. A mensagem a utilizar para pedir o identificador e a excepção a lançar (pertencente ao package `erp.app.exception`) caso o identificador não corresponda a um objecto conhecido (excepto no processo de registo ou caso indicado em contrário) depende do tipo de entidade em questão:

Entidade	Pedido a apresentar	Exceção a lançar se desconhecido
Empresa	<code>erp.app.company.Prompt.companyId()</code>	<code>UnknownCompanyKeyException</code>
Produto	<code>erp.app.product.Prompt.productId()</code>	<code>UnknownProductKeyException</code>
Cliente	<code>erp.app.client.Prompt.clientId()</code>	<code>UnknownClientKeyException</code>
Compra	<code>erp.app.purchase.Prompt.purchaseId()</code>	<code>UnknownPurchaseKeyException</code>

Existe ainda um caso particular: sempre que uma operação se refira a um produto no contexto de uma empresa e essa empresa não disponibilize esse produto, deve ser lançada a exceção `erp.app.exception.ProductNotSuppliedException` (a empresa e o produto existem, mas não estão relacionados). Existem ainda outros casos particulares que usam pedidos específicos não apresentados nesta tabela. Nestes casos, o pedido específico a utilizar será indicação na descrição do comando em causa. Note-se que estas exceções não devem ser utilizadas no código do domínio da aplicação como explicado na secção 3.

Vários comandos pedem ao utilizador que escolha um valor de um conjunto fechado (tipo de produto, categoria, estratégia de divisão, modalidade de transporte, serviço adicional e critério de pesquisa). Estas escolhas devem ser pedidas através de campos de opção (`Command.addOptionField()`, `Form.addOptionField()` ou `Form.requestOption()`), sendo o conjunto de opções obtido do domínio da aplicação através da fachada (`ErpManager`). Desta forma, a introdução de uma nova estratégia, modalidade, serviço ou critério não implica qualquer alteração aos comandos.

4.1 Formatos de apresentação

Os formatos usados pelos vários comandos são os seguintes. Todos os campos são separados por `_`.

Formato de apresentação de uma empresa:

```
id, nome
```

Formato de apresentação de um produto:

```
id_tipo_nome_categoria_preço_base_preço_recomendado_prazo_base_unidades_totais_informação_adicional
```

O tipo é *NACIONAL* ou *IMPORTADO*. As unidades totais correspondem à soma dos inventários do produto em todas as empresas que o disponibilizam. Para produtos nacionais não existe informação adicional; para produtos importados, a informação adicional corresponde ao país de origem e à taxa alfandegária. Exemplos de apresentação de produtos:

1	_	NACIONAL	_	Farinha	_	ALIMENTAR	_	15.00	_	15.00	_	3	_	173
2	_	IMPORTADO	_	Telemovel	_	ELECTRONICA	_	200.00	_	260.00	_	3	_	2_China_30.0
3	_	NACIONAL	_	Camisa	_	VESTUARIO	_	45.00	_	58.50	_	2	_	0

Formato de apresentação da disponibilização de um produto por uma empresa:

```
idEmpresa, -, idProduto, -, unidades, -, factor, -, preço de venda
```

O factor não é um preço: é apresentado tal como resulta da conversão de um double para texto (por exemplo, 0.9, 1.15 ou 2.0). O mesmo vale para a taxa alfandegária de um produto importado (30.0, 12.5). Exemplos de apresentação dos produtos disponibilizados por uma empresa:

$$\begin{array}{r} 2_1_150_0.9_13.50 \\ 2_2_2_1.2_312.00 \end{array}$$

Formato de apresentação de um cliente:

```
email, - nome, - gasto, total
```

O email é o identificador do cliente. O valor apresentado no fim corresponde ao total gasto pelo cliente em compras. Exemplos de apresentação de clientes:

Formato de apresentação resumido de uma compra:

id_ _emailCliente_ _idEmpresa_ _data_ _preço_total_dos_itens_ _custo_das_encomendas_ _custo_total

Formato de apresentação resumido de uma encomenda:

idCompra_ _descrição_do_transporte_ _data_prevista_de_entrega_ _custo_ _lista_de_identificadores_de_produtos

A descrição do transporte é a indicação da modalidade seleccionada, seguida das chaves dos serviços adicionais seleccionados, separadas por + (por exemplo, EXPRESSO+EMBRULHO+CARTAO_OFERTA). As chaves devem ser compostas pela ordem inversa pela qual os serviços foram adicionados à encomenda. Exemplos de apresentação de encomenda utilizando o formato resumidos:

1_ _EXPRESSO+EMBRULHO_ _5_ _10.00_ _3
1_ _EXPRESSO+EMBRULHO_ _6_ _15.00_ _1

4.2 Menu Principal

As acções deste menu permitem gerir a salvaguarda do estado da aplicação, ver e alterar a data actual e abrir submenus. A lista completa é a seguinte: **Abrir**, **Guardar**, **Mostrar data actual**, **Avançar data actual**, **Menu de Gestão de Empresas**, **Menu de Gestão de Produtos**, **Menu de Gestão de Clientes** e **Menu de Gestão de Compras**. Por omissão, as mensagens de diálogo para este menu estão definidos em `erp.app.main.Prompt` e `erp.app.main.Message`.

Os comandos que vão concretizar as funcionalidades deste menu já estão parcialmente concretizados nas várias classes do package `erp.app.main`: `DoOpenFile`, `DoSaveFile`, `DoDisplayDate`, `DoAdvanceDate`, `DoOpenMenuCompanies`, `DoOpenMenuProducts`, `DoOpenMenuClients` e `DoOpenMenuPurchases`.

4.2.1 Salvaguarda do estado actual

O conteúdo da aplicação (que representa o estado relevante actualmente em memória da aplicação) pode ser guardado para posterior recuperação (via serialização Java: `java.io.Serializable`). Na leitura e escrita do estado da aplicação, devem ser tratadas as excepções associadas. A funcionalidade é a seguinte:

Abrir – Carrega os dados de uma sessão anterior a partir de um ficheiro previamente guardado (ficando este ficheiro associado à aplicação, para futuras operações de salvaguarda). Pede-se o nome do ficheiro a abrir (utilizando a cadeia de caracteres devolvida por `Prompt.openFile()`). Caso ocorra um problema na abertura ou processamento do ficheiro, deve ser lançada a excepção `FileOpenFailedException`. A execução bem sucedida desta opção substitui toda a informação da aplicação. Quando se abandona uma aplicação com modificações não guardadas (porque se abre outra e existem alterações não guardadas), deve perguntar-se ao utilizador se quer guardar a informação actual antes de a abandonar, através de `Prompt.saveBeforeExit()` (a resposta pode ser obtida via `Form.confirm()` ou criando um novo `Form` com um campo do tipo `booleano`)).

Guardar – Guarda o estado actual da aplicação no ficheiro associado. Se não existir associação, pede-se o nome do ficheiro a utilizar ao utilizador (utilizando a cadeia de caracteres devolvida por `Prompt.newSaveAs()`), ficando a aplicação associada a este ficheiro. Não é executada nenhuma acção se não existirem alterações desde a última salvaguarda.

Note-se que a opção **Abrir** não suporta a leitura de ficheiros de texto (estes apenas são utilizados na inicialização da aplicação). A opção **Sair** **nunca** guarda o estado da aplicação, mesmo que existam alterações.

4.2.2 Mostrar data actual

A data actual do sistema é apresentada utilizando a mensagem devolvida por `Message.currentDate()`.

4.2.3 Avançar data actual

O número de dias a avançar é pedido utilizando a mensagem devolvida por `Prompt.daysToAdvance()`. O valor indicado deve ser positivo, caso contrário, a operação não tem efeito.

Além da data, o sistema deve desencadear as notificações correspondentes a cada um dos dias percorridos^{1.7}.

4.2.4 Gestão e consulta de dados da aplicação

A gestão e consulta de dados da aplicação são realizadas através das seguintes opções:

Menu de Gestão de Empresas – Abre o menu de gestão de empresas e operações associadas.

Menu de Gestão de Produtos – Abre o menu de gestão de produtos e operações associadas.

Menu de Gestão de Clientes – Abre o menu de gestão de clientes e operações associadas.

Menu de Gestão de Compras – Abre o menu de gestão de compras e operações associadas.

4.3 Menu de Gestão de Empresas

Este menu permite efectuar operações sobre as empresas registadas na aplicação. A lista completa é a seguinte: **Registar empresa, Mostrar empresa, Mostrar empresas, Alterar inventário de um produto, Mostrar produtos de uma empresa, Mostrar clientes de uma empresa.**

Os comandos deste menu já estão parcialmente concretizados nas classes do package `erp.app.company`: `DoRegisterCompany`, `DoShowCompany`, `DoShowCompanies`, `DoChangeProductInventory`, `DoShowCompanyProducts`, `DoShowCompanyClients`. Por omissão, as mensagens de diálogo para este menu estão definidos nas entidades `Prompt` e `Message` presentes no package `erp.app.company`.

4.3.1 Registar empresa

Pede o nome (`Prompt.companyName()`). O registo bem sucedido deve ser assinalado através da mensagem devolvida por `Message.registrationSuccessful()`; caso contrário, é lançada a excepção `CompanyRegistrationFailedException`. Note-se que a atribuição do identificador da empresa é automática.

4.3.2 Mostrar empresa

É pedido o identificador da empresa, sendo apresentadas as informações sobre essa empresa, no formato de apresentação de uma empresa (ver secção §4.1).

4.3.3 Mostrar empresas

Apresenta informações sobre todas as empresas, ordenando-as pelos seus identificadores. O formato de apresentação de cada empresa é o descrito na subsecção anterior.

4.3.4 Alterar inventário de um produto

Altera o inventário e factor multiplicativo de um produto numa empresa, permitindo também que uma empresa passe a disponibilizar um produto já registado no sistema, de acordo com as regras indicadas na secção 1.2.1. São pedidos o identificador da empresa, o identificador do produto, uma quantidade (`Prompt.amountToUpdate()`) e o factor multiplicativo do preço recomendado (`Prompt.priceFactor()`, um valor real). O sucesso desta operação é assinalado através da mensagem devolvida por `Message.supplySuccessful()`. Se a alteração for inválida, então a operação não deve ter qualquer efeito e deve ser apresentada a mensagem devolvida por `Message.cannotChangeInventory()`.

4.3.5 Mostrar produtos de uma empresa

É pedido o identificador da empresa e apresentam-se os produtos que essa empresa disponibiliza, ordenados pelos identificadores dos produtos, no formato de apresentação da disponibilização de um produto por uma empresa (ver secção §4.1).

4.3.6 Mostrar clientes de uma empresa

É pedido o identificador da empresa e apresentam-se os clientes que já interagiram com essa empresa, ordenados pelos seus identificadores, no formato de apresentação de um cliente (ver secção §4.1).

4.4 Menu de Gestão de Produtos

Este menu apresenta as operações disponíveis sobre produtos. A lista completa é a seguinte: `Registrar produto`, `Mostrar produto`, `Mostrar produtos`, e `Efectuar pesquisa`. Os comandos que concretizam estas operações já estão parcialmente concretizados nas classes do package `erp.app.product`: `DoRegisterProduct`, `DoShowProduct`, `DoShowProducts`, e `DoPerformSearch`. Por omissão, as mensagens de diálogo para este menu estão definidos em `erp.app.product.Prompt` e `erp.app.product.Message`.

4.4.1 Registrar produto

Regista um produto no registo global de produtos. São pedidos, por esta ordem: o tipo de produto (`Prompt.productType()`), o nome (`Prompt.productName()`), o preço base (`Prompt.basePrice()`), a categoria (`Prompt.productCategory()`) e o prazo de entrega base (`Prompt.baseDelay()`).

Se o tipo indicado for `IMPORTADO`, são ainda pedidos o país de origem (`Prompt.originCountry()`) e a taxa alfandegária (`Prompt.customsTax()`). O registo bem sucedido é assinalado através da mensagem devolvida por `Message.registrationSuccessful()`; caso contrário, é lançada a excepção `ProductRegistrationFailedException`. Note-se que a atribuição do identificador do produto é automática e que um produto registado não é, por si só, disponibilizado por qualquer empresa.

4.4.2 Mostrar produto

É pedido o identificador do produto. Se o produto existir, é apresentado no formato de apresentação de um produto (ver secção §4.1).

4.4.3 Mostrar Mostrar produtos

Apresenta informações sobre todos os produtos, ordenando-os pelos seus identificadores. O formato de apresentação de cada produto é o utilizado na subsecção anterior.

4.4.4 Efectuar pesquisa

Esta opção realiza uma procura sobre o registo de produtos. São pedidos o critério de pesquisa (`Prompt.searchCriterion()`) e um termo (`Prompt.searchTerm()`). Os critérios iniciais são: `NOME`, `CATEGORIA` e `ORIGEM` (país de origem, que apenas os produtos importados possuem). Se se indicar pesquisa por origem, apenas os produtos importados podem verificar o critério de pesquisa. Como resultado, deve ser apresentada a lista dos produtos encontrados pela pesquisa, ordenados por ordem crescente do seu identificador, utilizando o formato descrito na secção 4.4.2. O termo de pesquisa é comparado sem distinção entre letras maiúsculas e minúsculas. Caso não sejam encontrados produtos, não deve ser produzido qualquer resultado.

4.5 Menu de Gestão de Clientes

Este menu apresenta as operações disponíveis sobre clientes. A lista completa é a seguinte: `Registrar cliente`, `Mostrar cliente`, `Mostrar clientes` e `Mostrar notificações do cliente`. Os comandos que concretizam estas operações já estão parcialmente concretizados nas classes do package `erp.app.client`: `DoRegisterClient`, `DoShowClient`, `DoShowClients` e `DoShowClientNotifications`. Por omissão, as mensagens de diálogo para este menu estão definidas nas entidades `Prompt` e `Message` do package `erp.app.client`.

4.5.1 Registrar cliente

Pede o nome e o endereço de correio electrónico do cliente a registar utilizando as cadeias de caracteres devolvidas por `Prompt.clientName()` e `Prompt.clientEmail()`, respectivamente. O registo bem sucedido é assinalado através da mensagem devolvida por `Message.registrationSuccessful()`; caso contrário, é lançada a excepção `ClientRegistrationFailedException`. Note-se que o endereço de correio electrónico é o identificador do cliente: registar um endereço já conhecido falha, em vez de criar um segundo cliente.

4.5.2 Mostrar cliente

É pedido o identificador do cliente, sendo apresentadas as informações sobre esse cliente, no formato de apresentação de um cliente (ver secção §4.1).

4.5.3 Mostrar clientes

Apresenta informações sobre todos os clientes, ordenando-os lexicograficamente pelo nome. Caso existam clientes com o mesmo nome, devem ser ordenados pelos seus identificadores (os endereços de correio electrónico). A ordenação realizada com base nos nomes e nas chaves dos clients é lexicográfica sem distinção entre maiúsculas e minúsculas. O formato para apresentar cada cliente é o descrito em 4.5.2.

4.5.4 Mostrar notificações do cliente

Este comando apresenta as notificações recebidas pelo cliente através do mecanismo de notificações por omissão. É pedido o identificador do cliente, sendo apresentadas as notificações enviadas pelo mecanismo de notificações por omissão para esse cliente, de acordo com o seguinte formato:

```
assunto:_descrição_da_encomenda
```

O assunto é *ENTREGA AMANHA* (a encomenda será entregue no dia seguinte) ou *ENTREGUE* (a encomenda foi entregue). A descrição da encomenda é a descrição resumida definida na secção §4.1. Exemplo de notificações para um dado cliente:

```
ENTREGA_AMANHA:_1_-_EXPRESSO+EMBRULHO_-_5_-_10.00_-_3
ENTREGUE:_1_-_EXPRESSO+EMBRULHO_-_6_-_15.00_-_1
```

As notificações de um cliente apenas devem ser apresentadas uma única vez. Após esta visualização, considera-se que o cliente fica sem notificações. As notificações devem ser apresentadas pela mesma ordem em que foram enviadas pelo sistema. Caso o cliente indicado não tenha notificações, não deve ser apresentado qualquer resultado.

4.6 Menu de Gestão de Compras

Este menu apresenta as operações relacionadas com compras e encomendas. A lista completa é a seguinte: *Efectuar compra*, *Mostrar compra* e *Mostrar compras*. As operações deste menu já estão parcialmente concretizadas nas classes da package `erp.app.purchase`, respectivamente: `DoPurchase`, `DoShowPurchase`, e `DoShowPurchases`. Por omissão, as mensagens de diálogo para este menu estão definidas nas entidades `Prompt` e `Message` do package `erp.app.purchase`.

4.6.1 Efectuar compra

Esta opção não efectua, por si só, qualquer compra: abre o menu de cabaz sobre o cabaz de um cliente numa empresa. São pedidos o identificador do cliente e o identificador da empresa. Se ambos existirem, é aberto o menu de cabaz (ver secção 4.7) sobre o cabaz desse cliente nessa empresa (criado vazio, se ainda não existir). Como o cabaz pertence ao cliente, sair do menu sem finalizar a compra preserva o seu conteúdo e as suas opções para uma visita posterior.

4.6.2 Mostrar compra

É pedido o identificador da compra e apresenta a compra em detalhe utilizando o seguinte formato:

```
id_emailCliente_-_idEmpresa_-_data_-_preço_total_dos_itens_-_custo_das_encomendas_-_custo_total
idCompra_-_descrição_do_transporte_-_data_prevista_de_entrega_-_custo
_idProduto_-_nome_-_quantidade_-_preço_unitário_-_total_da_linha
...
```

Na primeira linha apresenta-se a informação da compra; de seguida, apresentam-se as suas encomendas em detalhe — o cabeçalho de cada encomenda (sem a lista de identificadores de produtos) seguido das suas linhas, indentadas com dois espaços e ordenadas pelos identificadores dos produtos. Cada linha tem o mesmo formato das linhas de um cabaz; o preço unitário apresentado é o que foi fixado no momento da compra, e não o preço de venda actual. As encomendas são apresentadas por ordem crescente da data prevista de entrega e, em caso de empate, pelos identificadores dos produtos na primeira posição da lista correspondente. Exemplo de apresentação de uma compra:

```
1_-_obiwan@jedi.org_-_1_-_1_-_94.35_-_25.00_-_119.35
1_-_EXPRESSO+EMBRULHO_-_5_-_10.00
3_-_Camisa_-_1_-_64.35_-_64.35
1_-_EXPRESSO+EMBRULHO_-_6_-_15.00
1_-_Farinha_-_2_-_15.00_-_30.00
```

4.6.3 Mostrar compras

Apresenta informações sobre todas as compras, ordenando-as pelos seus identificadores, no formato de apresentação de uma compra (apenas a linha da compra, sem as encomendas, ver secção §4.1)).

4.7 Menu de Cabaz

Este menu apresenta as operações sobre um cabaz: é aberto sobre o cabaz em causa, que é o alvo de todos os seus comandos: A lista completa é a seguinte: Alterar produto no cabaz, Mostrar cabaz, Definir estratégia de divisão em encomendas, Definir modalidade de transporte, Adicionar serviço adicional e Finalizar compra. Algumas opções só estão disponíveis quando fazem sentido, não sendo apresentadas nos restantes casos: (i) a opção **Finalizar compra** só está disponível no menu se o cabaz tiver, pelo menos, uma linha e se já tiver sido escolhida a modalidade de transporte; (ii) a opção **Definir modalidade de transporte** só está disponível se existir pelo menos uma modalidade de transporte definida.

As operações deste menu já estão parcialmente concretizadas nas classes do package `erp.app.basket`, respectivamente: `DoChangeProduct`, `DoShowBasket`, `DoSetSplitStrategy`, `DoSetModality`, `DoAddService` e `DoCheckout`. Todos derivam de `erp.app.basket.BasketCommand`, cujo alvo (*receiver*) é o cabaz. Por omissão, as mensagens de diálogo para este menu estão definidos em `erp.app.basket.Prompt` e `erp.app.basket.Message`.

4.7.1 Alterar produto no cabaz

Altera a quantidade de um produto no cabaz. São pedidos o identificador do produto (`Prompt.productToChange()`) e a quantidade a alterar (`Prompt.amountToChange()`), que pode ser negativa e actualiza a quantidade do produto indicado de acordo com as regras definidas na secção 1.4. O sucesso da operação é assinalado com a mensagem `Message.productChanged()`, indicando a nova quantidade resultante. Se a alteração remover o produto do cabaz, é apresentada a mensagem `Message.productRemoved()`. Se o produto não estiver no cabaz e a quantidade indicada não for positiva, nada é alterado e é apresentada a mensagem `Message.productNotInBasket()`. Se a empresa não tiver exemplares suficientes para satisfazer a quantidade total resultante, o cabaz não é alterado e é apresentada a mensagem `Message.notEnoughInventory()`. Se a empresa não disponibilizar o produto indicado, é lançada a excepção `ProductNotSuppliedException`.

4.7.2 Mostrar cabaz

Apresenta o cabaz: na primeira linha, o seu resumo; de seguida, as suas linhas, ordenadas pelos identificadores dos produtos e indentedas com dois espaços. O formato utilizado para apresentar um cabaz é o seguinte:

```
emailCliente_ _idEmpresa_ _estratégia_ _descrição_do_transporte_e_serviços_ _preço_total_dos_itens
 _idProduto_ _nome_ _quantidade_ _preço_unitário_ _total_da_linha
... ..
```

Exemplo de apresentação de um cabaz

```
obiwan@jedi.org_ _1_ _PRAZO_ _EXPRESSO+EMBRULHO_ _94.35
 _1_ _Farinha_ _2_ _15.00_ _30.00
 _3_ _Camisa_ _1_ _64.35_ _64.35
```

Note-se que os preços apresentados são os praticados pela empresa no momento em que o cabaz é mostrado: só são fixados ao finalizar a compra. A descrição do transporte acompanha o estado da escolha: *SEM TRANSPORTE* enquanto faltar a modalidade de transporte. Exemplos de descrição do transporte de um cabaz: *SEM TRANSPORTE*, *EXPRESSO*, *NORMAL+EMBRULHO*.

4.7.3 Definir estratégia de divisão em encomendas

É pedida a estratégia (`Prompt.splitStrategy()`), que determina em quantas encomendas a compra será dividida. Por omissão, o cabaz usa a estratégia de encomenda única (UNICA).

4.7.4 Definir modalidade de transporte

É pedida a modalidade (`Prompt.modality()`), que será usada por todas as encomendas da compra. Por omissão, o cabaz não tem modalidade escolhida.

4.7.5 Adicionar serviço adicional

É pedido o serviço (`Prompt.serviceToAdd()`), que passa a ser aplicado a todas as encomendas da compra. É possível adicionar repetidamente um serviço.

4.8 Finalizar compra

Transforma o cabaz numa compra e fecha o menu de cabaz. O efeito no modelo é o descrito na secção 1.4. Se a compra não puder ser finalizada por faltarem exemplares de algum dos produtos, é lançada a excepção `erp.app.exceptions.NotEnoughInventoryException` e o menu de cabaz não é fechado. Em caso de sucesso, a compra resultante é apresentada seguindo o formato definido para apresentar uma compra com detalhe na secção 4.6.2 e o menu de cabaz é fechado.

5 *Leitura de Dados a Partir de Ficheiros Textuais*

Além das opções de manipulação de ficheiros descritas na secção §4.2.1, é possível iniciar a aplicação com um ficheiro de texto especificado pela propriedade Java com o nome `import`. Quando se especifica esta propriedade, a aplicação é povoada com os objectos correspondentes ao conteúdo do ficheiro indicado, sendo possível registar empresas, clientes e produtos, e definir as tarifas das modalidades de transporte e as disponibilizações de produtos por parte das empresas. Cada linha deste ficheiro textual descreve uma dada entidade a carregar no estado inicial do sistema. Pode assumir que não existem entradas mal-formadas nestes ficheiros e que os campos de texto não podem conter o carácter `:`, que os preços base e as tarifas são números inteiros e que a taxa alfandegária e o factor são números reais. Pode ainda assumir que o ficheiro especifica sempre pelo menos uma modalidade de transporte. Cada linha do ficheiro a processar segue um dos seguintes formatos:

```
RATE:modalidade:custo:dias
COMPANY:nome
CLIENT:nome:email
NATIONAL:nome:preçoBase:categoria:prazo
IMPORTED:nome:preçoBase:categoria:prazo:paisOrigem:taxa:prazoInternacional
SUPPLY:idEmpresa:idProduto:unidades:factor
```

Note-se que o registo de um produto não indica qualquer empresa nem qualquer inventário: essas são propriedades da disponibilização, descrita por entradas próprias (*SUPPLY*), em que unidades é o número de exemplares que a empresa tem disponíveis e factor o multiplicador (real) que aplica ao preço recomendado. As entradas *SUPPLY* referem empresas e produtos já registados em linhas anteriores. A codificação dos ficheiros a ler é garantidamente UTF-8. Sugere-se a utilização do método `String.split` para o processamento preliminar das linhas do ficheiro de texto. De seguida descreve-se o conteúdo de um possível ficheiro textual que representa um determinado estado inicial do sistema:

```
RATE:NORMAL:3:3
RATE:EXPRESSO:8:1
COMPANY:Moageira_Nacional
COMPANY:Importadora_Atlantico
CLIENT:Obi-Wan_Kenobi:obiwan@jedi.org
CLIENT:Ahsoka_Tano:ahsoka@jedi.org
NATIONAL:Farinha:15:ALIMENTAR:3
IMPORTED:Telemovel:200:ELECTRONICA:3:China:30
NATIONAL:Camisa:45:VESTUARIO:2
SUPPLY:1:1:23:1.0
SUPPLY:2:1:150:0.9
SUPPLY:2:2:2:1.2
SUPPLY:1:3:4:1.1
```

Note-se que o programa **nunca** produz ficheiros com este formato, apenas lê ficheiros com este formato.

6 *Execução do Programa e Testes Automáticos*

É possível verificar automaticamente o resultado correcto do programa. Cada teste é constituído por dois ou três ficheiros de texto:

- um com extensão `.in` e que representa o utilizador, ou seja, vai conter os dados de entrada a serem processados pelo programa;
- um com extensão `.out` e que representa o resultado esperado da execução do programa para o teste em causa;

- um com extensão `.import` e que representa o estado inicial do sistema. Este ficheiro é opcional e pode não fazer parte de um teste.

Dado um teste constituído pelos ficheiros `test.import`, `test.in` e `test.out`, é possível verificar automaticamente o resultado correcto do programa. Note-se que pode ser necessária a definição apropriada da variável de ambiente `CLASSPATH` (ou da opção equivalente `-cp` do comando `java`), para localizar as classes do programa, incluindo a que contém o método correspondente ao ponto de entrada da aplicação (`erp.app.App.main`). As propriedades são tratadas automaticamente pelo código de apoio.

```
java -cp -Dimport=test.import -Din=test.in -Dout=test.outhyp erp.app.App
```

Caso o teste não tenha o ficheiro de *import*, então a forma de executar o teste é semelhante ao anterior, não especificando a propriedade *import*. Assumindo que aqueles ficheiros estão no directório onde é dado o comando de execução, o programa produz o ficheiro de saída `test.outhyp`. Em caso de sucesso, os ficheiros da saída esperada (`test.out`) e obtida (`test.outhyp`) devem ser iguais. A comparação pode ser feita com o comando:

```
diff -b test.out test.outhyp
```

Este comando não deve produzir qualquer resultado quando os ficheiros são iguais. Note-se, contudo, que cada teste apenas verifica um determinado comportamento da aplicação.

7 Notas de Concretização

Tal como indicado neste documento, algumas classes fornecidas como material de apoio, são de uso obrigatório e não podem ser alteradas. Outras dessas classes são de uso obrigatório e têm de ser alteradas.